

01背包问题

今年过去的“双十一”，你有薅到羊毛吗？

每年的双十一，会有各种促销活动，比如“满 300 元减 80 元”。假如你女朋友的购物车中有 n 个 ($n > 100$) 想买的商品，她希望从里面选几个，在凑够满减条件的前提下，让选出来的商品价格总和最大程度地接近满减条件 ($300 \leq \text{price} \leq 380$)，这样就可以极大限度地“薅羊毛”。作为一名“聪明”的程序员，你有想过编程帮她搞定吗？

要想高效地解决这个问题，就要用到我们今天讲的 **01 背包问题** (0-1 Knapsack Problem)。首先记住一点，**01 背包问题** 不是一个问题，而是一类动态规划问题，很多动态规划问题都可以抽象成 01 背包问题。

问题描述

01 背包问题：给定 n 件不可分割的物品和一个背包。物品 i 的重量是 $w[i]$ ，其价值为 $v[i]$ ，背包的容量为 c 。问应如何选择装入背包中的物品，使得装入背包中的物品在不超过背包容量的情况下总价值最大？

在选择装入背包的物品时，对每种物品 i 只有两种选择，即装入背包 (1) 和不装入背包 (0)。不能将物品装入背包多次，也不能只装入商品的一部分（商品不可分割）。这就是经典的 **0-1 背包问题**。

问题的 **形式化描述** 是，给定 $c > 0, w_i > 0, v_i > 0, 1 \leq i \leq n$ ，要求找出一个 n 元 0-1 向量 $(x_1, x_2, \dots, x_n)$ ， $x_i \in \{0, 1\}, 1 \leq i \leq n$ ，使得

$$\sum_{i=1}^n w_i x_i \leq c$$

，而且

$$\sum_{i=1}^n w_i x_i$$

达到最大。因此，0-1 背包问题是一个特殊的整数规划问题：

$$\max \sum_{i=1}^n v_i x_i \begin{cases} \sum_{i=1}^n w_i x_i \leq c \\ x_i \in \{0, 1\}, 1 \leq i \leq n \end{cases}$$

01背包问题简化版

为了理解的方便，我们可以将原 01 背包问题简化一下：

给定 n 件不可分割的物品和一个背包。物品 i 的重量是 $w[i]$ ，背包的容量为 c 。问应如何选择装入背包中的物品，请问装入背包的所有物品的最大重量是多少？

问题的形式化描述是，给定 $c > 0, w_i > 0, 0 \leq i \leq n$ ，要求找出一个 n 元 0-1 向量 $(x_1, x_2, \dots, x_n)$ ， $x_i \in \{0, 1\}, 1 \leq i \leq n$ ，求

$$\sum_{i=1}^n w_i x_i \leq c$$

的最大值。因此，0-1背包问题是一个特殊的整数规划问题：

$$\max \sum_{i=1}^n w_i x_i \leq c$$

考虑一个简单输入示例：

背包容量 $c = 10$

物品个数 $n = 5$

物品重量为 $wt[] = \{2, 2, 4, 6, 5\}$

我们将题目中的物品价值暂时去掉了，这样更方便我们掌握动态规划和 01 背包问题，我们先考虑用递归对问题进行解决。

暴力递归就是枚举物品集合的所有子集，并计算每一个子集的总重量，最后选择与背包的总容量 c 最接近的子集即为最优解。

考虑物品的最优子集，对于每一个物品 i 均有下面两种情况。

- 1、物品 i 包含在最优子集中
- 2、物品 i 不包含在最优子集中。

如果添加第 n 个物品后，背包的重量超过了总容量 c ，则第 n 个物品就不能装入背包；否则，则可以将第 n 个物品装入背包。

回顾一下递归的三要素：

第一：明确你写的递归函数想要干什么，即函数功能；

第二：找到递归的结束条件；

第三：找出函数的等价关系式。

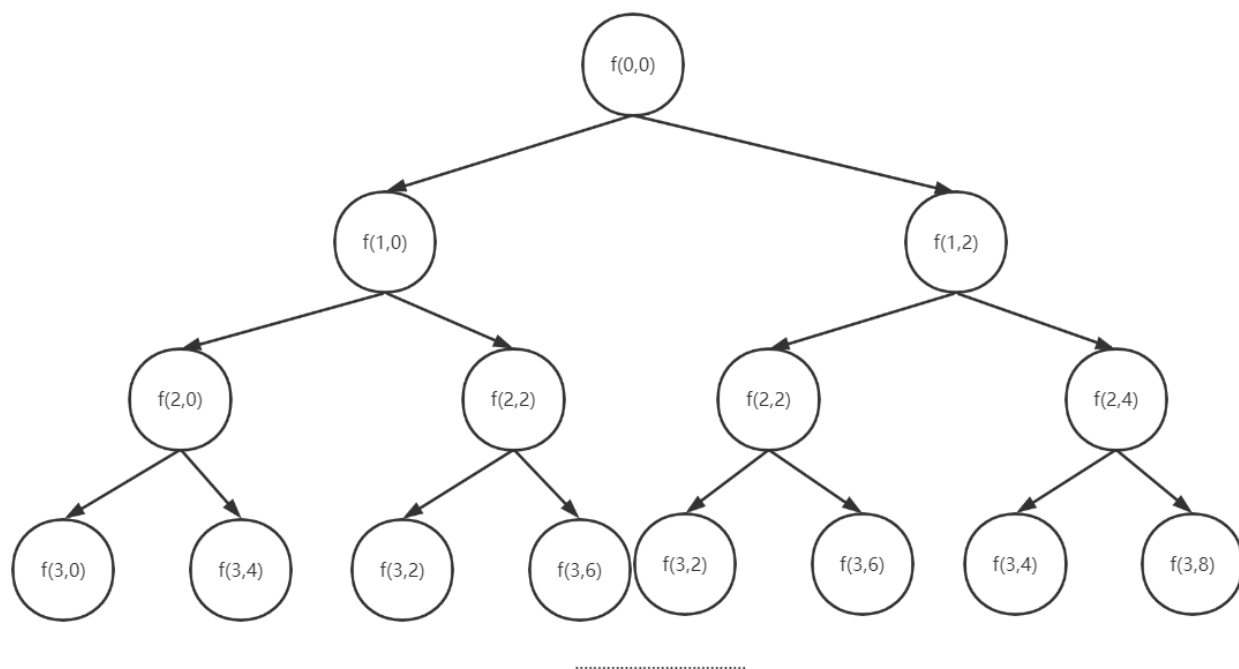
```
class Knapsack{
    private int maxW = Integer.MIN_VALUE; // 保存背包中可容纳的最大重量
    // w 表示当前已经装进背包的物品的总重量；i表示考察到了哪个物品 i
    private int[] wt = {2,2,4,6,5}; //表示每一个物品的重量
    private n = 5; // n 表示物品总数
    private c = 9; // c 背包容量
    //第一要素：函数功能，决定是否将第 i 个物品装入背包，从而获得最大重量
    public void Knapsack(int i, int w){
        //递归结束条件
        if(w == c || i == n){ // w == c 表示装满了，i == n 物品考察完了
            if(w > maxW){
                maxW = w;
            }
            return;
        }
    }
}
```

```

// 等价关系式，装 or 不装
Knapsack(i+1, w); // 选择不装第 i 个物品
if(w + wt[i] <= c){
    Knapsack(i+1, w + wt[i]); // 选择装第 i 个物品。
}
}
}

```

递归回溯算法的代码虽然看着简洁明了，但是其时间复杂度比较高，是指数级别的。为了更清晰地看到其低效的原因，老规矩，画出递归树。我们依旧使用输入示例，画出递归树如下：



递归树中的每一个结点表示一种状态，我们用 (i, w) 来表示，其中， i 表示将要决策的第 i 个物品是否装入背包， w 表示当前背包中物品的总重量。比如， $(3, 8)$ 表示我们要决策的物品第 3 个物品（重量为 4）是否装入背包，决策后，将其装入背包，当前背包的重量为 8； $(3, 4)$ 则表示我们当前要决策的物品是第 3 个物品，在决策后，不将其装入背包，当前背包的重量为 4。

显而易见，递归树中有很多子问题被重复计算，比如图中的 $f(2, 2)$ 和 $f(3, 4)$ 均被重复计算了两次。

要对这些重复计算的结点进行剪枝，我们就可以使用 DP Table 和备忘录方法。

“备忘录”方法，就是将已经计算好的子问题的解 $f(i, w)$ 保存起来，当再次计算到重复的 $f(i, w)$ 时，直接从备忘录中取出来用就行了，不用再递归计算，这样就有效地避免重复计算，达到剪枝效果。

```

class Knapsack{
    private int maxW = Integer.MIN_VALUE;

    private int[] wt = {2,2,4,6,5};
    private n = 5;
    private c = 9;
    private boolean[][] mem = new boolean[5][10]; //备忘录，默认均为 false
    public void Knapsack(int i, int w){

```

```

//递归结束条件
if(w == c || i == n){ // w == c 表示装满了, i == n 物品考察完了
    if(w > maxW){
        maxW = w;
    }
    return;
}
if(mem[i][w]) return; // 重复状态

mem[i][w] = true; // 记录状态

Knapsack(i+1, w); // 选择不装第 i 个物品
if(w + wt[i] <= c){
    Knapsack(i+1, w + wt[i]); // 选择装第 i 个物品。
}
}
}

```

备忘录方法是自顶向下的方法，与递归的结构一致，且其在性能方面和动态规划的基本一致。但我们主要学习的是动态规划，所以进入今日的主角。

我们把原问题的整个求解过程分为 n 个阶段，每个阶段会决策一个物品是否放到背包中。每个物品决策（放入或者不放入背包）完之后，背包中的物品的重量会有多种情况，也就是说，会达到多种不同的状态，对应到递归树中，就是有很多不同的节点。

我们把每一层重复的状态（节点）合并，只记录不同的状态，然后基于上一层的状态集合，来推导下一层的状态集合。我们可以通过合并每一层重复的状态，这样就保证每一层不同状态的个数都不会超过 c 个（ c 表示背包的承载重量），也就是例子中的 9。于是，我们就成功避免了每层状态个数的指数级增长。

我们用一个二维数组 `dp[n][w+1]`，来记录每层可以达到的不同状态。

第 0 个（下标从 0 开始编号）物品的重量是 2，要么装入背包，要么不装入背包，决策完之后，会对应背包的两种状态，背包中物品的总重量是 0 或者 2。我们用 `dp[0][0] = 1` 和 `dp[0][2] = 1` 来表示这两种状态。这实际上就是我们原问题里面的 n 元 0-1 向量 $(x_1, x_2, \dots, x_n)$, x_i 属于 $\{0, 1\}$, $1 \leq i \leq n$ 。即 `(1, 0, 1, 0, 0, 0, 0, 0, 0)`。

对于第 1 个物品的重量也是 2，基于之前的背包状态，在这个物品决策完之后，不同的状态有 3 个，背包中物品总重量分别是 0(0+0), 2(0+2 or 2+0), 4(2+2)。我们用 `dp[1][0] = 1`，`dp[1][2] = 1`，`dp[1][4] = 1` 来表示这三种状态。即 `(1, 0, 1, 0, 1, 0, 0, 0, 0)`。

以此类推，直到决策完所有的物品后，整个 **DP Table** 就算都计算好了。我们可以自己计算一遍，就可以得到下面的 DP Table 了，我们只需要在决策完最后一件物品的最后一行，找出值为 1 的最接近 c （这里是 9）的值，就是可以装入背包中物品的总重量的最大值。

可获得的最大重量		0	1	2	3	4	5	6	7	8	9
物品编号	0	1	0	1	0	0	0	0	0	0	0
	1	1	0	1	0	1	0	0	0	0	0
	2	1	0	1	0	1	0	1	0	1	0
	3	1	0	1	0	1	0	1	0	1	0
	4	1	0	1	0	1	1	1	1	1	1

DP Table

最大重量

实现代码其实就是填表的整个过程，你能自己手填出此表，看代码简直轻而易举。

```
class Knapsack{
    private int maxW = Integer.MIN_VALUE;

    private int[] wt = {2,2,4,6,5};
    private n = 5;
    private c = 9;

    public int Knapsack(int[] wt, int n, int c){
        boolean[][] dp = new boolean[n][c+1]; // 默认为false, 即为0

        dp[0][0] = true; // 初始状态
        dp[0][wt[0]] = true; // 第一行数据，也就是起始状态，特殊处理。

        //从决策第二个物品开始，自底向上 DP Table
        for(int i = 1; i < n; i++) {
            for(int j = 0; j <= c; j++){
                if(dp[i-1][j]){ // 不装入第 i 个物品
                    dp[i][j] = dp[i-1][j];
                }
            }
            //装入第 i 个物品
            for(int k = 0; k <= c - wt[i]; k++){
                if(dp[i-1][k]){
                    dp[i][k + wt[i]] = true;
                }
            }
        }

        for(int i = c; i >= 0; i--){
            if(dp[n-1][i]) return i;
        }
    }
}
```

```
        return 0;
    }
}
```

这就是基于 DP Table 的动态规划的自底向上的填表过程，把问题分解成多个阶段，每个阶段对应一个决策，我们记录每一个阶段所有可达的状态集合，然后用前面阶段已经得到的状态来推导当前状态集合，动态地向前推进，这就是动态规划的由来，虽然还挺贴切，但是还是 DP Table 来的舒服！

已知暴力递归，枚举所有可能的组合的时间复杂度为指数级别的 $O(n^2)$ ，基于 DP Table 的动态规划的时间复杂度为 $O(n \times c)$ ，其中 n 表示物品的个数，而 c 表示可以背包的总容量（Capacity）。

但是聪明的你也一定发现了一个问题，刚才的代码中的 DP Table 是一个二维数组，而且我们事实上，当我们决策第 i 个物品是否装入背包的状态时，仅使用到了其前一个状态 $i - 1$ 的状态值，所以我们只需要一个大小为 $c + 1$ 的一维 DP Table 就可以解决这个问题。

我们可以仔细观察一下上面代码中注释 **不装入第 i 个物品** 的情况，拿到不就是将第 $i - 1$ 个物品的状态直接拷贝到第 i 个物品对应的状态数组中吗？比如，初始状态（即决策了第 0 个物品之后的状态集合）为：

	0	1	2	3	4	5	6	7	8	9
dp[] =	1	0	1	0	0	0	0	0	0	0

现在要决策是否将第 1 个物品（重量也为 2）是否装入背包，我们会考虑两种情况：

将物品不装入背包，则此时第 1 个物品的状态集合与第 0 个物品的状态集合一样，按照之前的二维 `dp` 数组，会将其拷贝一次，但是现在 `dp` 数组变成了一维，我们有必要再拷贝一次吗？

将物品装入背包，此时我们直接对一维的 `dp[]` 数组进行修改不就可以了吗？

所以要将原来使用的二维 DP Table 变成 一维的，只需要考虑装入的情况，然后直接对一维的 DP Table 进行修改即可。简单来说，你可以像下图这样理解，原来一个二维的 `dp[][]` 会记录所有阶段的状态值，而现在一维的 `dp[]` 只记录当前决策的物品的所有状态值：



```
class Knapsack{
    private int maxW = Integer.MIN_VALUE;

    private int[] weight = {2,2,4,6,5};
    private n = 5;
    private c = 9;

    public int Knapsack(int[] weight, int n, int c){
        boolean[] dp = new boolean[c+1]; // 默认为false, 即为0
        dp[0] = true; // 初始状态
        dp[weight[0]] = true; // 第一行数据, 也就是起始状态, 特殊处理。
        //从决策第二个物品开始, 自底向上 DP Table
        for(int i = 1; i < n; i++) {
```

```

        //装入第 i 个物品
        for(int j = c - weight[i]; j >= 0 ; --j){
            if(dp[j]){
                dp[j+weight[i]] = true;
            }
        }
    }

    for(int i = c; i >= 0; i--){
        if(dp[i]) return i;
    }
    return 0;
}
}

```

这里一定要注意内层循环控制变量 `j` 是从 `c - weight[i]` 开始由大到小进行逆序遍历，因为 `j` 从小到大顺序遍历的话，会出现 for 循环重复计算和值被覆盖的情况。

我们以最后一次更新（即决策第 4 个物品是否装入背包）为例进行说明，更新前（第 3 个物品的状态集合）`dp[]` 数组的状态为：

3	1	0	1	0	1	0	1	0	1	0
---	---	---	---	---	---	---	---	---	---	---

`dp[] = DP Table`

现在更新第 4 个物品（重量为 5）是装入背包后的状态集合，我们先看看从小到大进行处理的会发生什么？

`j = 0` 时，`dp[0] = 1`，则将 `dp[j + weight[i]] = dp[0+5] = true`；

`j = 1` 时，`dp[1] = 0`，跳过；

`j = 2` 时，`dp[2] = 1`，则将 `dp[2 + 5] = dp[7] = 1`；

`j = 3` 时，`dp[3] = 0`，跳过；

`j = c - weight[i] = 9 - 5 = 4` 时，`dp[4] = 1`，则将 `dp[4 + 5] = dp[9] = 1`

；

这里似乎看起来不出来什么，但是当物品的数目比较多，背包的容量比较大的时候，就会出现某一个值 `dp[k]` 依赖与它前面的某一个状态 `dp[x]` 的情况，我们为了避免使用更新后的状态 `dp[x]` 去更新 `dp[k]` 的情况才从大到小进行计算的。

虽然对于这个例子，你看到正序和逆序都不会影响最终的结果，但我希望你铭记：采用一维 DP Table 对状态进行保存并更新时，内层循环一定要从大到小进行更新！

现在让我们回到原始的 0-1 背包问题。

01背包问题

我们再来回忆一下他的形式化的描述。

问题的 **形式化描述** 是，给定 $c > 0, w_i > 0, v_i > 0, 1 \leq i \leq n$ ，要求找出一个 n 元 0-1 向量 $(x_1, x_2, \dots, x_n)$. $x_i \in \{0, 1\}, 1 \leq i \leq n$ ，使得

$$\sum_{i=1}^n w_i x_i \leq c$$

，而且

$$\sum_{i=1}^n w_i x_i$$

达到最大。因此，0-1背包问题是一个特殊的整数规划问题：

$$\max \sum_{i=1}^n v_i x_i \begin{cases} \sum_{i=1}^n w_i x_i \leq c \\ x_i \in (0, 1), 1 \leq i \leq n \end{cases}$$

最优子结构

设 $(y_1, y_2, \dots, y_n)$ 是所给 0-1 背包问题的一个最优解，则 $(y_1, y_2, \dots, y_n)$ 是下面相应子问题的一个最优解：

$$\max \sum_{i=2}^n v_i x_i \begin{cases} \sum_{i=2}^n w_i x_i \leq c - w_1 y_1 \\ x_i \in (0, 1), 1 \leq i \leq n \end{cases}$$

（反证法）如若不然，设 $(z_1, z_2, \dots, z_n)$ 是上述子问题的一个最优解，而 $(y_1, y_2, \dots, y_n)$ 不是它的最优解，由此可知

$$\sum_{i=2}^n v_i z_i > \sum_{i=2}^n v_i y_i$$

且

$$w_1 y_1 + \sum_{i=2}^n w_i z_i \leq c$$

因此

$$w_1 y_1 + \sum_{i=2}^n v_i z_i > \sum_{i=1}^n v_i y_i$$

这说明 $(y_1, z_1, z_2, \dots, z_n)$ 是所给 0-1 背包问题的一个更优解，从而 $(y_1, y_2, \dots, y_n)$ 不是所给 0-1 背包问题的最优解。此为矛盾。所以 0-1 背包问题具有最优子结构性质。

递归关系

设所有 01 背包问题的子问题为：

$$\max \sum_{i=1}^n v_k x_k \begin{cases} \sum_{k=i}^n w_k x_k \leq c \\ x_i \in (0, 1), i \leq k \leq n \end{cases}$$

子问题的最优值为 $m(i,j)$ ，即 $m(i,j)$ 是背包容量为 j ，可选物品为 $i, i+1, \dots, n$ 时 0-1 背包问题的最优值。由 0-1 背包问题的最优子结构性质，可以建立计算 $m(i,j)$ 的递归式如下：

$$m(i,j) = \begin{cases} \max(m(i+1,j), m(i+1,j-w_i) + v_i), & j \geq w_i \\ m(i+1,j), & 0 \leq j < w_i \end{cases}$$

背包容量 j 大于等于物品的重量，则把物品 i 装入背包。

$$m(i,j) = \begin{cases} \max\{m(i+1,j), m(i+1,j-w_i) + v_i\} & j \geq w_i \\ m(i+1,j) & 0 \leq j < w_i \end{cases}$$

背包容量 j 小于物品的重量，则把不装入物品 i 。

$$m(n,j) = \begin{cases} v_n & j \geq w_n \\ 0 & 0 \leq j < w_n \end{cases}$$

边界条件，当背包的容量为 j ，第 n 个物品的重量小于等于 j ，也就是这个物品可以装入背包，则 $m(n,j)$ 的最优值为 v_n ，否则为 0 。

请不要拒绝数学推导和动态规划转移方程的数学表达，这才是真正的动态规划，只是我们为了应付面试、笔试才在网上看到各种模板解题套路，真正的套路是自己内化而成的！

递归法

有了上面的状态转移方程，不难写出下面的递归代码：

```
class Knapsack {

    // 返回两个整数的较大值
    static int max(int a, int b)
    {
        return (a > b) ? a : b;
    }

    // 第一要素：明确你写的递归函数想要干什么
    // 函数功能：计算可以放入容量为 w 的背包的最大价值
    // W:背包容量，wt[]:每一个物品的重量，val[]:每一个物品所对应的价值
    static int knapSack(int W, int wt[], int val[], int n)
    {
        // 第二要素：找到递归的结束条件（一定要考虑全面）
        // 背包容量 w 为 0 或者物品个数为 0，可获得的价值必然为0
        if (n == 0 || W == 0)
            return 0;

        // 如果第 n 个商品的重量大于背包容量 w，则该商品不能包含在最优解中
        if (wt[n - 1] > W){
            return knapSack(W, wt, val, n - 1);
        }
    }
}
```

```

    }
    // 第三要素：找出函数的等价关系式：取 (1) 和 (2) 的较大值
    // (1) val[n - 1] + knapSack(W - wt[n - 1], wt, val, n - 1) 包含第 n 个
物品
    // (2) knapSack(W, wt, val, n - 1) 不包含第 n 个物品
    else{
        return max(val[n - 1] + knapSack(W - wt[n - 1], wt, val, n - 1),
knapSack(W, wt, val, n - 1));
    }
}
}
}

```

可以看到递归函数重复计算了子问题的解。如下所示的递归树， $K(1,1)$ 被计算了两次。

注意：**0-1**背包问题如果用我们前面提到的示例输入解释比较复杂，我们选择了一个更简单的输入，让大家理解**0-1**背包问题！

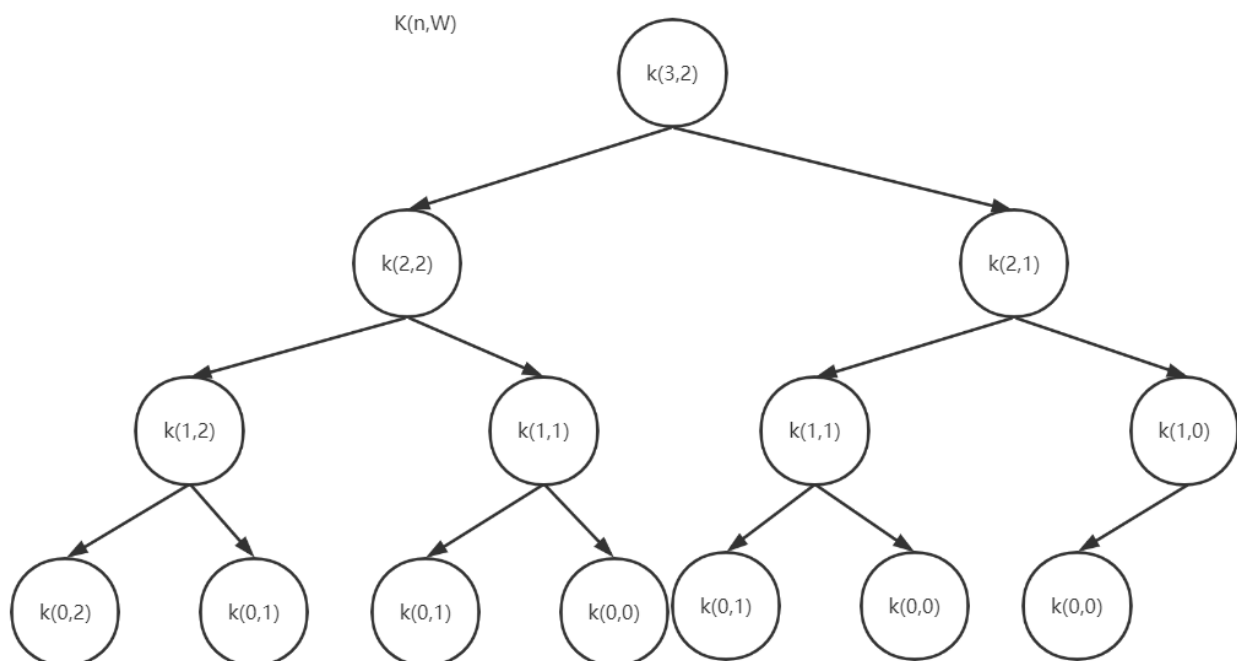
0-1 背包问题输入示例：

$wt[] = \{1, 1, 1\}$ // 物品的重量

$W = 2$ // 背包的容量

$val[] = \{10, 20, 30\}$ // 物品对应的价值

下图中的递归树表示的是 $K(i, W)$ 表示背包重量为 W ，可选择物品为 $i, i+1, \dots, n$ 时 0-1 背包问题的最优值。



其实，抛开了物品的价值，这个递归树就和前面讲的简化版本一个样，没啥稀奇的。

动态规划方法

紧接着模仿简化的 0-1 背包问题，创建一个二维的 `DP[][]` 数组，用来记录每层可以达到的不同状态（决策第 i 个物品，背包从容量 1 到 W 的所有状态值）。不过这里数组存储的值不再是布尔类型了，而是当前状态对应的最大总价值。

```
class Knapsack {

    static int max(int a, int b)
    {
        return (a > b) ? a : b;
    }

    // 返回容量为 w 的背包可容纳的最大价值
    static int knapSack(int W, int wt[], int val[], int n)
    {
        int i, w;
        int K[][] = new int[n + 1][W + 1];

        // 自底向上构建 DP Table
        for (i = 0; i <= n; i++)
        {
            for (w = 0; w <= W; w++)
            {
                if (i == 0 || w == 0){
                    K[i][w] = 0;
                }
                else if (wt[i - 1] <= w){
                    K[i][w] = max(val[i - 1] + K[i - 1][w - wt[i - 1]],
                                   K[i - 1][w]);
                }
                else{
                    K[i][w] = K[i - 1][w];
                }
            }
        }

        return K[n][W];
    }

    public static void main(String args[])
    {
        int val[] = new int[] { 6, 10, 12 };
        int wt[] = new int[] { 1, 2, 3 };
        int W = 5;
        int n = val.length;
        System.out.println(knapSack(W, wt, val, n));
    }
}
```

```
}
```

同样的道理，我们可以将二维的 `dp[][]` 数组用一个一维的数组保存起来，将空间复杂度降低

```
class KnapSack{

    static int KnapSack(int val[], int wt[], int n, int W)
    {
        //dp[i] 存储容量为 "i" 时背包的最大价值
        int[] dp = new int[W+1];

        //将 dp[] 初始化为 0
        Arrays.fill(dp, 0);

        // 迭代所有物品
        for(int i = 0; i < n; i++){
            //从大到小遍历dp数组并更新（和之前一样）
            for(int j = W; j >= wt[i]; j--){
                dp[j] = Math.max(dp[j] , val[i] + dp[j - wt[i]]);
            }
        }
        return dp[W];
    }

    public static void main(String[] args)
    {
        int val[] = {6, 10, 12}, wt[] = {1, 2, 3}, W = 5, n = 3;
        System.out.println(KnapSack(val, wt, n, W));
    }
}
```

购物车薅羊毛

掌握 0-1 背包问题之后，用动态规划薅羊毛，岂不是很简单了？

每年的双十一，会有各种促销活动，比如“满 300 元减 80 元”。假如你女朋友的购物车中有 n 个 ($n > 100$) 想买的商品，她希望从里面选几个，在凑够满减条件的前提下，让选出来的商品价格总和最大程度地接近满减条件 ($300 \leq \text{price} \leq 380$)，这样就可以极大限度地“薅羊毛”。作为一名“聪明”的程序员，你有想过编程帮她搞定吗？

从原问题提取出我们的输入输出

输入：

`items[] = {56, 188, 66, 88, 190}` // n 件商品的价格 (相当于 0-1 背包问题中物品的重量)

`W = 380` // 最多可接受的凑单价格，相当于 0-1 背包问题中背包的容量。

输出：

可以参与凑单的商品列表 `dp`，这个过程就是向上回溯输出的过程。

```
class Double11Collage{
    private static final int DISCOUNT = 80;
    public static void double11Collage(int[] items, int n, int price) {
        int W = price + DISCOUNT;
        boolean[][] dp = new boolean[n][W];

        dp[0][0] = true;
        dp[0][items[0]] = true;

        for (int i = 1; i < n; ++i) { // 动态规划
            for (int j = 0; j <= W; ++j) { // 不购买第 i 个商品
                if (dp[i-1][j] == true) dp[i][j] = dp[i-1][j];
            }
            for (int j = 0; j <= W-items[i]; ++j) { // 购买第 i 个商品
                if (dp[i-1][j]==true) dp[i][j+dp[i]] = true;
            }
        }
        int j;
        for (j = price; j < W; ++j) {
            if (dp[n-1][j] == true) break; // 输出结果大于等于拼单价的最小值
        }
        if (j == W) return; // 没有可行解
        for (int i = n-1; i >= 1; --i) { // i 表示二维数组中的行, j 表示列
            if (j-items[i] >= 0 && dp[i-1][j-items[i]] == true) {
                System.out.print(items[i] + " "); // 购买这个商品
                j = j - items[i];
            } // else 没有购买这个商品, j 不变。
        }
        if (j != 0){
            System.out.print(items[0]);
        }
    }
}
```

求可以凑成拼单价格的最低价格，不就和 0-1 背包的代码一样吗？这里不能再考虑使用空间复杂度优化的动态规划了，我们要保存决策过程中的每一个状态，从而由决策完最后一件商品之后的最优解向上回溯，找到最优解所涉及的商品并打印出来。

状态 `(i, j)` 只有可能从 `(i-1, j)` 或者 `(i-1, j-items[i])` 两个状态推导过来。所以，我们就检查这两个状态是否是可达的，也就是 `dp[i-1][j]` 或者 `dp[i-1][j-items[i]]` 是否 `true`。

如果 `dp[i-1][j]` 可达，就说明我们没有选择购买第 `i` 个商品，如果 `dp[i-1][j-items[i]]` 可达，那就说明我们选择了购买第 `i` 个商品。我们从中选择一个可达的状态（如果两个都可达，就随意选择一个），然后，继续向上回溯，看其他商品是否有选择购买并打印输出。

总结

就买东西而言，动态规划在女朋友前不堪一击！期待你将其优化，证明一波学习比恋爱更有趣！

正题：0-1背包问题是一类问题，可以说，让你在理解的基础之上将上面解决0-1背包的三种代码（递归、动态规划、空间优化的 DP）背下来也不为过，所以多看几遍，在理解的基础上把 0-1 背包的代码多默写几遍！